

SQLのメモ

- [publishについて](#)

CHAPTER 1

publish

publish IN ('検索値 1','検索地 2') AND price >= 3000 はpriceが3000以上のフィールド名publishの検索地はどのレコード化探す

```
SELECT
    isbn,
    title,
    price
FROM
    books
WHERE
    publish IN ('日経BP','翔泳社')
AND
    price >= 3000
;
```

ORDER BY

ソート条件を指定する

※ORDER BYが一番最後に処理される

```
ORDER BY
    answer1 ASC
| ソートキー | 並び順 |
```

ASC ... 昇順 DESC... 降順

ORDER BYは, を使って第二キーを設定できる

```
ORDER BY
    answer1 ASC,
    answer2 DESC
```

数字を使ってフィールドを指定する

※フィールドは1から始まる

```
ORDER BY  
  4 DESC;
```

LIMIT

特定のレコードのみを抽出する

AVG()などの関数を使ってみる

グループごとに平均を求める

```
SELECT  
  publish,  
  AVG(price)  
FROM  
  books  
GROUP BY  
  publish  
;
```

SELECTの所に書いてもいいもの

NG

- グループ内で共通ではないタイトル 理由は集計したときにどのタイトルを表示すればいいかわからなくなるから

OK

- グループ化キーに指定した列(共通タイトル)
- 集計関数
- 定数

GROUP BYでグループの中をさらにグループ化

```
GROUP BY  
  publish,  
  title
```

このようにしてグループの中をtitleというグループに再度分けることができる

AVGのポイント

AVG(タイトル)がダメな理由

AVG(タイトル)はそのタイトルのフィールド列にNULLがあったときに、数えられない仕組みになっているから

関数を使った処理

```
SELECT
  -- 文字をくっつける関数CONCAT
  CONCAT('ISBN',isbn) AS 'ISBNコード',
  title AS '書名',
  DATE_FORMAT(publish_date, '%Y年%m月%d日') AS '刊行日'
FROM
  books
;
```

GROUP BYの後にHAVING

FROM -> WHERE -> GROUP BY -> HAVING -> SELECTの順で実行される

HAVINGは、GROUP BYで持ってきた値から条件を絞り込む

この構文はanswer1を平均してprefectureでグループ化した中で、平均値が2未満のものだけ取り出す

```
SELECT
  AVG(answer1) AS '評価平均'
FROM
  quest
GROUP BY
  prefecture
HAVING
  AVG(answer1) < 2
;
```

CASE文

```
SELECT
  referer,
  COUNT(*) AS 'カウント数',
  CASE
    WHEN COUNT(*) >= 50 THEN 'A',
    WHEN COUNT(*) >= 10 THEN 'B',
    ELSE 'C'
  END AS 'ランク'
FROM
  access_log
GROUP BY
  referer
;
```

CASEをSELECT以外で

CASE文をHAVINGで使うこともある。

```
HAVING COUNT(*) = SUM(CASE WHEN check_flag = '完了'...)
```

データベースの掟

- 「UNIQUE制約（他のデータと重複してはいけない）」

主キーは重複してはダメであり、名前なども重複しているとエラーになる。

```
やるなら  
太郎 1  
太郎 2  
みたいな？
```

- 「非NULL制約（NULLであってはいけない）」

カラムの分割

データの意味が壊れないほど分解したほうが良い

例 1 「氏名 -> 苗字と名前」

例 2 「県 -> 県と町と所在地」

なぜ分ける？

SQLはくっつけるのは簡単だが、分解するのは難しいから

非正規系の問題点

```
商品名 1  
どっさりカイロ  
もっさりカイロ
```

このような場合があったとき、どっさりカイロが本物なのか、もっさりカイロが本物なのか分からなくなってしまい、エラーになる。

第一正規化

注文番号、商品番号を加えることで、商品名や単価や購入数を特定できる

[改善点]

商品の購入数がテーブル定義によって制限されることはなくなった

[問題点]

- ・値の更新によって矛盾が生じることがある
- ・行の削除によって情報が損失することがある
- ・注文があるまですべての情報はデータベースに登録できない

●第1正規化後のテーブル

注文番号	顧客番号	顧客名	顧客住所
P00001	USR001	山田奈美	千葉県小金井市福山 0-0-00
P00002	USR002	薄井浩	静岡県鎌ヶ谷市静波 9-8-98
P00003	USR003	日尾晃子	埼玉県新田原市三次 1-1-11
P00004	USR001	山田奈美	千葉県小金井市福山 0-0-00

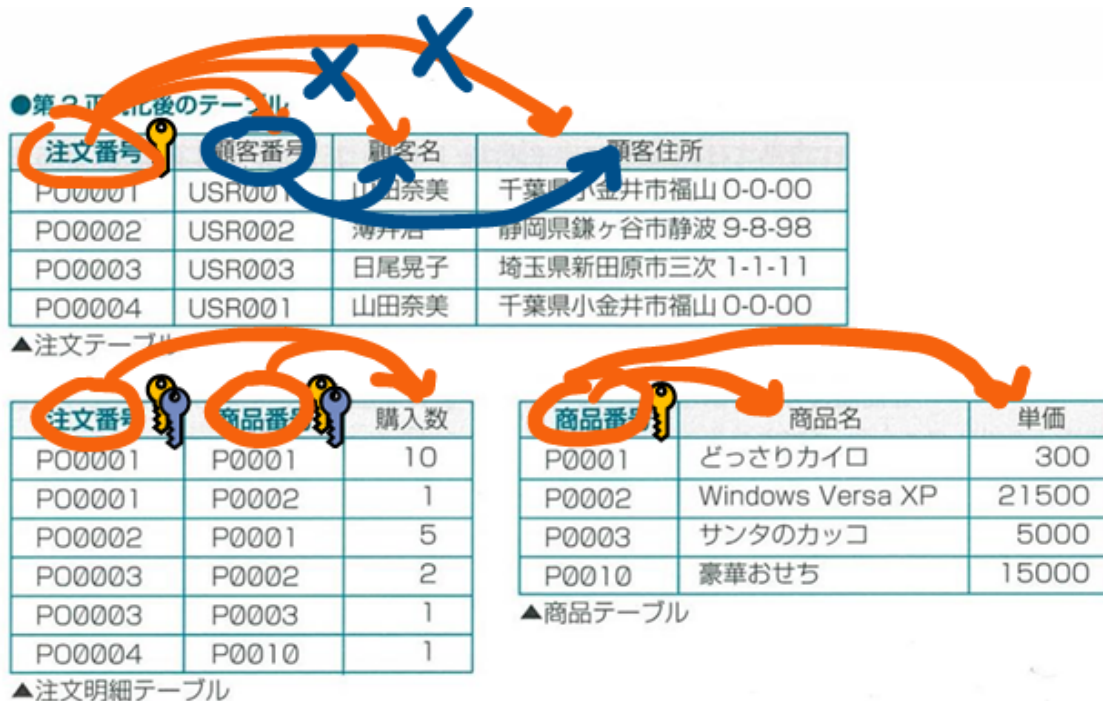
▲注文テーブル

注文番号	商品番号	商品名	単価	購入数
P00001	P0001	どっさりカイロ	300	10
P00001	P0002	Windows Versa XP	21500	1
P00002	P0001	どっさりカイロ	300	5
P00003	P0002	Windows Versa XP	21500	2
P00003	P0003	サンタのカッコ	5000	1
P00004	P0010	豪華おせち	15000	1

▲注文明細テーブル

第二正規化

テーブルを分割することで、どっさりカイロ、もっさりカイロ現象がなくなる



正規化まとめ

正規化は分割することが多いので、第二正規化にもどすことがある。

```

FROM
    employee AS e    -- ここはemployeeとdepartをくっつけてる
INNER JOIN
    depart AS d      -- eやdを使って名前を省略している
ON
  
```

```
e.depart_id = d.depart_id
-- ここでキーを結び付けている
```

例

AI00001	相沢	聡	アイザワ	サトシ	1	部長	S02	NULL	2012-02-24	0	S02	総務部
DA00001	大門	一郎	ダイモン	イシロウ	1	担当	J01	AI00001	2011-08-10	0	J01	人事部
FU00001	藤井	雄太郎	フジイ	ユウタロウ	1	主任	E01	SE00001	2011-04-10	0	E01	第一営業部
FU00002	藤岡	幸太郎	フジオカ	コウタロウ	1	担当	E01	FU00001	2011-06-12	0	E01	第一営業部
HA0001	速水	和幸	ハヤミ	ユウコ	1	部長	E01	NULL	2007-02-03	1	E01	第一営業部
HA0002	寒山	俊輔	ハヤマ	シュンスケ	1	アシスタント	E01	FU00002	2012-12-11	0	E01	第一営業部
KA00001	川口	裕子	カウグチ	ユウコ	2	アシスタント	S01	KI0001	2012-12-28	1	S01	経営企画部
KA00002	加藤	昭雄	カトウ	アキオ	1	アシスタント	S01	KI0001	2011-11-11	0	S01	経営企画部
KA00003	神田	佐知子	カンダ	サチコ	1	アシスタント	J01	KA00003	2012-01-28	0	J01	人事部
KI00001	木村	一郎	キムラ	イチロウ	1	担当	S01	TA00001	2011-07-12	0	S01	経営企画部
NA00001	中澤	康代	ナカザワ	ヤスヨ	2	アシスタント	S02	TO00002	2012-04-17	1	S02	総務部
NO00001	野上	利江	ノガミ	リエ	2	アシスタント	S02	TO00002	2012-06-30	0	S02	総務部
SA00001	佐藤	智子	サトウ	トモコ	2	部長	S01	NULL	2010-02-01	0	S01	経営企画部
SE00001	妹尾	春樹	セノオ	ハルキ	1	課長	E01	TE00001	2010-12-10	0	E01	第一営業部
SI00001	清水	春子	シミズ	ハルコ	2	主任	J01	SI00001	2011-12-11	0	J01	人事部
TA00001	田中	真由子	タナカ	マユコ	2	主任	S01	YA00001	2011-05-26	0	S01	経営企画部
TA00002	田辺	正一	タナベ	ショウイチ	1	アシスタント	E01	HA0002	2012-12-19	0	E01	第一営業部
TE00001	寺岡	陽一	テラオカ	ヨウイチ	1	部長	E01	HA0001	2009-12-11	0	E01	第一営業部
TI00001	近田	兎子	チカオ	アキコ	2	課長	S02	AI00001	2012-03-01	0	S02	総務部
TO00001	遠山	真一	トヤマ	シンイチ	1	課長	J01	UE00001	2010-12-16	0	J01	人事部
TO00002	戸川	よしみ	トガム	ヨシミ	2	担当	S02	TU00001	2012-04-01	0	S02	総務部
TU00001	津村	知美	ツムラ	カズミ	2	主任	S02	TI00001	2012-03-16	0	S02	総務部
UE00001	上田	一也	ウエダ	カズヤ	1	部長	J01	NULL	2011-11-12	0	J01	人事部
YA00001	山田	奈美	ヤマダ	ナミ	2	課長	S01	SA00001	2012-10-15	0	S01	経営企画部

ここでretiredまでがemployeeでそこから右側がdepartになる

フィールドの取得

```
SELECT
  e.l_name,
  e.f_name,
  d.depart_name,
  e.class
FROM
  employee AS e INNER JOIN depart AS d
ON
  e.depart_id = d.depart_id
WHERE
  -- 役職がなかった場合は1となる
  e.retired <> 1
ORDER BY
  e.depart_id ASC,
  e.s_id ASC;
;
```

外部結合

```
SELECT
  CONCAT(e.l_name,e.f_name),
  AVG(t.work_time)
FROM
  employee AS e
LEFT JOIN
  time_card AS t
```

```

ON
    e.s_id = t.s_id
GROUP BY
    e.s_id
ORDER BY
    AVG(t.work_time) DESC
;

```

イメージとしては->フィールド名1,2とフィールド名3をくっつける

```

+-----+
フィールド名1 | フィールド名2 | フィールド名3 |
+-----+
xxxxxxxxxx   | xxxxxxxxxxxxxx | xxxxxxxxxxx   |
xxxxxxxxxx   | xxxxxxxxxxxxxx | xxxxxxxxxxx   |
xxxxxxxxxx   | xxxxxxxxxxxxxx | NULL          |
xxxxxxxxxx   | xxxxxxxxxxxxxx | NULL          |
+-----+

```

RIGHT JOINは右側のテーブルを基準にして左側のテーブルに右側のフィールドのデータがなかったら、NULLとなる

LEFT JOINは上の左側バージョン

OUTER JOINについて

INNER JOINは論理積で考える -> NULLがどちらかにあれば表示しない

OUTER JOINは論理和で考える -> NULLがどちらかにあっても必ず表示する

同一テーブル内のデータを結合する

以下のようなものを作れば、リスト構造ができる

```

SELECT
    *
FROM
    contents AS cc
    INNER JOIN
    contents AS cn
ON
    -- ここで同じテーブル内のidを交じ合わせる
    cc.next_id = cn.c_id
;

```

c_id	c_title	next_id	c_id	c_title	next_id
A001	SQLについて	A011	A011	データの取得1	A012
A011	データの取得1	A012	A012	カラムの選択	A013
A012	カラムの選択	A013	A013	WHERE句の記述	A014
A013	WHERE句の記述	A014	A014	NULLと空白の扱い	A015
A014	NULLと空白の扱い	A015	A015	あいまい検索	A016
A015	あいまい検索	A016	A016	複数条件の連結（論理演算子）	A017
A016	複数条件の連結（論理演算子）	A017	A017	重複レコードの除去	A021
A017	重複レコードの除去	A021	A021	データの取得2	A022
A021	データの取得2	A022	A022	順位付け	A023
A022	順位付け	A023	A023	ソート	A024
A023	ソート	A024	A024	グルーピング	A025
A024	グルーピング	A025	A025	グルーピング結果の絞り込み	A026
A025	グルーピング結果の絞り込み	A026	A026	クロス集計	A027
A026	クロス集計	A027	A027	サブクエリー	A028
A027	サブクエリー	A028	A028	演算子による集計	A029
A028	演算子による集計	A029	A029	複数表の結合	A031
A029	複数表の結合	A031	A031	更新系SQL	A032
A031	更新系SQL	A032	A032	レコードを1件挿入する	A033
A032	レコードを1件挿入する	A033	A033	サブクエリで複数レコードを挿入する	A034
A033	サブクエリで複数レコードを挿入する	A034	A034	全レコード一律の更新を行う	A035
A034	全レコード一律の更新を行う	A035	A035	特定行の更新を行う	A036
A035	特定行の更新を行う	A036	A036	特定レコードの削除を行う	A037
A036	特定レコードの削除を行う	A037	A037	トランザクション	A041
A037	トランザクション	A041	A041	データベースの作成	A042
A041	データベースの作成	A042	A042	データベースの作成／削除	A043
A042	データベースの作成／削除	A043	A043	制約条件	A044
A043	制約条件	A044	A044	テーブルの作成／削除	A045
A044	テーブルの作成／削除	A045	A045	テーブル情報の変更	A046
A045	テーブル情報の変更	A046	A046	インデックスの作成／削除	A047

同一テーブルを三つ結合

()の中が最初の結合、その次に()の外の設定で結合することで三つ結合が可能

```
SELECT
    b.title,
    a.name,
    b.publish_date
FROM
    (
        books AS b
        INNER JOIN
        author_books AS ab
        ON
            b.isbn = ab.isbn
    )
    INNER JOIN
    author AS a
ON
    ab.author_id = a.author_id
WHERE
    b.publish = '日経BP'
ORDER BY
    b.publish_date DESC
;
```

赤色がひとつめの結合

黄色が二つ目の結合

•

```
1  SELECT
2      b.title,
3      a.name,
4      b.publish_date
5  FROM
6      (
7          books AS b
8          INNER JOIN
9          author_books AS ab
10         ON
11         b.isbn = ab.isbn
12     )
13     INNER JOIN
14     author AS a
15     ON
16     ab.author_id = a.author_id
17 WHERE
18     b.publish = '日経BP'
19 ORDER BY
20     b.publish_date DESC
21 ;
```

サブクエリ

二つのクエリを同時に実行する

```
SELECT
    name,
    age,
    answer1,
    answer2
FROM
    quest
WHERE
    -- ここで毎回、新鮮な平均年齢を得ることができる -> 直接書く手間が省ける
    age > (SELECT AVG(age)
           FROM quest)
ORDER BY
    answer1 ASC
;
```

NOT INについて

NOT IN演算子は、IN演算子の反対である。

IN(a,b,c) ... この中に、指定したものがあるか？
NOT IN(a,b,c) ... この中の、指定したものではないか？

```
SELECT
  s_id,
  s_name
FROM
  shop
WHERE
  s_id NOT IN (
    SELECT
      s_id
    FROM
      sales
    WHERE
      s_date = '2012-12'
  )
;
```

ここで仮想的にテーブルを作る

以下のテーブルからサブクエリで抽出し、持ってくるイメージ

s_id	s_date	s_value
H0001	2012-11	9975
H0001	2012-12	9861
K0001	2012-11	10142
K0001	2012-12	11234
K0002	2012-11	10124
K0002	2012-12	10456
M0001	2012-11	10784
M0001	2012-12	11145
M0002	2012-11	12032
N0001	2012-11	9965
N0001	2012-12	11023

NOT EXISTS()について

NOT EXISTS()はEXISTSの反対で、存在していたらFalseになる。もしも存在していなかったらTrueとなり、実行される

※書き方の注意：NOT EXISTSはサブクエリの結果がFalseだったらTrueになるのでWHERE NOT EXISTSという風に、書かないといけない

```
SELECT
  l_name,
  f_name
FROM
  usr
WHERE NOT EXISTS ( -- 以下のクエリを実行して、rentalテーブルにuser_idが存在していなかったら取り出
```

```
す
SELECT
    *
FROM
    rental
WHERE
    rental.user_id = usr.user_id
);
```

CREATE VIEWについて

CREATE VIEWでメソッドのように呼び出しが行うことができる

```
CREATE VIEW
    view_sales
AS
SELECT
    s_id
FROM
    sales
WHERE
    s_date = '2012-12'
;

SELECT * FROM view_sales;
```

UNION句

二つのクエリの結果を純粹に結合する -> INNER JOINのもっと分かりやすいバージョン

```
SELECT
    u.l_name_kana,
    u.f_name_kana,
    'a'
FROM
    usr AS u
UNION --上のクエリと下のクエリをそのまま横に結合して表示
SELECT
    e.l_name_kana,
    e.f_name_kana,
    e.sex
FROM
    employee AS e
ORDER BY 1,2
```

UNION ALLについて

UNION ALLとは、重複したデータも全て表示する機能である

差集合と和集合 -> バージョンで動かないやつがたくさんある

EXCEPT演算子 -> 差集合

INTERSECT演算子 -> 積集合

挿入クエリ

そのテーブルの基準のフィールドの順に指定する

```
INSERT INTO
  usr
VALUES(
  'A200507',
  '鈴木',
  '徳次郎',
  'スズキ',
  'トクジロウ',
  '群馬県',
  '群馬市北町',
  '1-1-1',
  '040-999-9999',
  NULL
)
```

フィールドを最初から指定して挿入

```
INSERT INTO usr (
  user_id, --ここでどのフィールドに挿入するか決める
  l_name,
  f_name,
  l_name_kana,
  f_name_kana,
  tel
)
VALUES
(
  'B200507', --ここで上の指定したフィールドにこのデータを入れる
  '神田',
  '愛',
  'カンダ',
  'アイ',
  '040-888-8888'
)
```

※ここで指定しなかったフィールドにNULLを入れていいか、確認してから行う

INSERTとSELECTクエリを混ぜる

SELECTクエリで取り出した値を、usrテーブルに追記する

```
INSERT INTO usr (user_id, l_name, f_name, email)
SELECT
    s_id,
    l_name,
    f_name,
    'admin@wings.msn.to'
FROM
    employee
WHERE
    last_update >= '2012-01-01'
;
```

UPDATE命令

値の変更を加えることができる

```
-- booksテーブルのpriceフィールドのデータをすべて1.05倍する
UPDATE
    books
SET
    price = price * 1.05
;
```

条件を指定したレコードを更新する

```
-- idが3のデータを変更
UPDATE
    quest
SET
    answer1 = 3,
    answer2 = ' ',
    answered = NOW()
WHERE
    id = 3
;
```

DELETEクエリ

指定したレコードを削除する

```
-- s_dateが2012-11以下のレコードを消す
DELETE FROM
  sales
WHERE
  s_date <= '2012-11'
;
```

トランザクション処理

リクエストが二つ来た時に片方を処理して、その次にもう片方を処理するような処理

```
-- トランザクション開始
BEGIN;

-- データの追加
INSERT INTO books(isbn, title)
VALUES('4-0010-000X', 'SQL入門2');

-- booksテーブルの確認
SELECT
  isbn,
  title
FROM
  books
;
```

ここでのトランザクション処理の説明

トランザクションを開始する ... BEGIN;
以下のトランザクションを同時に実行する ... データの追加, booksテーブルの確認

クエリの実行後にROLLBACKコマンドでロールバックし、なかったことにする

ロールバック

すべての変更を取り消しにする

ロールバックが使えるのは、上にBEGIN;と書いてあるファイルだけ

```
# 直前のクエリ（更新や削除）をなかったことにする
ROLLBACK;
```

テーブル作成クエリの作り方

```
CREATE TABLE
quest
(
    id INT AUTO_INCREMENT, -- AUTO_INCREMENTでオート版バー型にできる
    name VARCHAR(100) NOT NULL, -- VARCHARで何文字までか指定
    name_kana VARCHAR(255) NOT NULL, -- NOT NULLでNULLの格納を禁止する
    sex VARCHAR(5) NOT NULL,
    prefecture VARCHAR(10) NOT NULL,
    age INT DEFAULT 0,
    answer1 INT NULL,
    answer2 TEXT NULL,
    answered DATETIME NOT NULL,
    PRIMARY KEY (id) -- ここで主キーは何かを設定する
);
```

主キーの別の書き方

```
id INT AUTO_INCREMENT PRIMARY KEY --最初から一行目に着けておく
```

テーブルの消し方

DROPコマンドでテーブルを削除できる

```
DROP TABLE テーブル名;
```

情報の出力

データベース一覧を表示

```
SHOW DATABASES;
```

テーブル一覧を表示

```
SHOW TABLES;
```

そのテーブルのフィールドと必要情報を一覧表示する

```
SHOW COLUMNS FROM テーブル名;
```

インデックス

検索効率を向上させるための仕組み

主キーを基準にするのではなく、データとなるフィールドの名前や日付を降順や昇順にすることによって、フルスキャンせずにレコードを検索することができる

CREATE INDEX

pub_date --publish_dateが主キーになるように変更をかけたインデックスと、publish_dateが主キーになるように変更をかけた二つのインデックスが作られる。

ON

```
books
(
    publish,
    publish_date
)
;
```

どんな仕組みなのか？

SELECT

*

FROM

books

WHERE

publish = '翔泳社' --ここではpublishが翔泳社になっているので->

;

-- 翔泳社が検索されるとき、主キー（isbn）をもとに、検索がかけられている。しかし、上のCREATE INDEXでインデックスを作成しておくことで、publishを主キーとするテーブルが裏に作成され、そのテーブルを上でのSELECTクエリが認識して扱う。その際に、直接publishを主キーとしているので、元のbooksテーブルを使うよりも早くなる

どんなインデックスが裏に存在しているのか確認する方法

```
SHOW INDEX FROM テーブル名
```

インデックスが使われないとき（自動認識される）

- IS NULL, IS NOT NULL, <>, LIKEなどの演算子を使うとき
- インデックス列に対して演算をしている

```
```sql
```

```
-- これはあり、右側で演算を行えば通る
```



```
price < 3000 * 1.05
...
```

- 複合インデックスの場合。そしてインデックス列がWHERE文で対象となっていない場合。

## ALTER テーブル

テーブルを変更更新してくれる機能。主に既存のテーブルにフィールドを追加したいときに使う。デフォルトはNULL

```
ALTER TABLE --ここで対象となるテーブル指定
 quest
ADD --どんなフィールドを追加するか
 last_update DATETIME -- DATETIMEはデータ型
AFTER --どのフィールドの後ろに追加するか指定
 answered
;
```

AFTERのところについて

**AFTERの部分でもし一番手前にフィールドを作りたい場合、FIRSTを使う**

```
ADD
 last_update DATETIME
FIRST
;
```

テーブルも消すことができる

```
ALTER TABLE
 quest
DROP -- ageフィールドを削除することができる
 age
;
```

## 特定の列の制約条件を変更

文字数の制限などの設定を変更する

**SQLiteでは制約条件の変更をすることができないため、最初から作り直さないといけない**

```
ALTER TABLE
 usr
MODIFY
 o_address VARCHAR(255) NULL --o_addressはもともとVARCHAR(100)だった
;
```

### 実行前

```
mysql> SHOW COLUMNS FROM usr;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| user_id | char(7) | NO | PRI | | |
| l_name | varchar(20) | YES | | NULL | |
| f_name | varchar(20) | YES | | NULL | |
| l_name_kana | varchar(100) | YES | | NULL | |
| f_name_kana | varchar(100) | YES | | NULL | |
| prefecture | varchar(15) | YES | | NULL | |
| city | varchar(20) | YES | | NULL | |
| o_address | varchar(100) | YES | | NULL | |
| tel | varchar(20) | YES | | NULL | |
| email | varchar(255) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

### 実行後

```
mysql> SHOW COLUMNS FROM usr;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| user_id | char(7) | NO | PRI | | |
| l_name | varchar(20) | YES | | NULL | |
| f_name | varchar(20) | YES | | NULL | |
| l_name_kana | varchar(100) | YES | | NULL | |
| f_name_kana | varchar(100) | YES | | NULL | |
| prefecture | varchar(15) | YES | | NULL | |
| city | varchar(20) | YES | | NULL | |
| o_address | varchar(255) | YES | | NULL | |
| tel | varchar(20) | YES | | NULL | |
| email | varchar(255) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

## フィールドの名前を変更する

```
ALTER TABLE
 old_name --ここで今までの既存のテーブル名
RENAME AS
 new_name --新しく名前変更するテーブル名
;
```

## ここから応用編

## ウィンドウ関数（別名:OLAP関数）

データベースを使ってリアルタイムデータ分析を行う処理のこと

例えば、市場分析、財務諸表作成、マネジメント計画作成など、ビジネスの現場でなくてはならないものである

```
<ウィンドウ関数>OVER (
 [PARTITION BY <列リスト>]
 OVER BY <ソート用列リスト> <ASCかDESCか>
)
```

### 例文

```
-- 順位付け関数を使った例
SELECT
 title,
 price,
 publish,
 RANK() OVER (
 PARTITION BY publish --publish列をグループ化する
 ORDER BY price DESC --そのグループでpriceの順位(DESC:降順)をつける
) AS 'ランキング'
FROM
 books
;
```

※PARTITION BYがなかった場合、すべてのレコードを基準にランキングを付ける

## 順位付けの方法 3 選

- RANK()

RANKで出力すれば、データが重複しても順位が一緒になる。そして、5位→7位など重複したレコード分順位を飛ばす

- DENSE\_RANK()

DENSE\_RANKで出力すれば、データが重複しても順位が一緒になる。5位→7位などと順位は飛ばされずに、そのまま順位付けをする

- ROW\_NUMBER()

ROW\_NUMBERで出力すれば、データが重複したら自動的に順位がつけられる。

※ウィンドウ関数はSELECT句以外では使わないようにしよう

title	price	publish	RANK	DENSE_RANK	ROW_NUMBER
SQL入門	5500	山田出版	1	1	1
PHPドリル	5100	山田出版	2	2	2
XML辞典	3300	翔泳社	3	3	3
XMLリファレンス	3200	日経BP	4	4	4
PHP5サンプル集	3000	秀和システム	5	5	5
PEAR入門	3000	翔泳社	5	5	6
SQLリファレンス	2500	日経BP	7	6	7
JSPリファレンス	1800	秀和システム	8	7	8
SQLプチブック	1600	日経BP	9	8	9
らくだの観察日記	1100	山田出版	10	9	10
フェレットの観察	1000	山田出版	11	10	11
ハムスターの観察	900	山田出版	12	11	12
かえるの観察日記	800	山田出版	13	12	13
あひるの観察日記	700	山田出版	14	13	14

14 rows in set (0.01 sec)

## 累計の方法

- SUM()

SUM(フィールド)で出力すれば、指定したフィールドを集計する

```
SELECT
 title,
 price,
 publish,
 -- SUMでは引数を渡してあげないといけない。price
 SUM(price) OVER (ORDER BY price DESC) AS '累計'
FROM
 books
;
```

## PRECEDINGを使って、直近の3行分の平均値を出す

ここ最近の調子的なものを把握するのに便利

```
-- AVGを使った集計と、PRECEDINGを使った例
SELECT
 title,
 price,
 publish,
 SUM(price) OVER (ORDER BY price DESC) AS '累計',
 AVG(price) OVER (ORDER BY price DESC
 ROWS 2 PRECEDING) AS '現在の平均'
FROM
```

```
books
;
```

title	price	publish	移動平均
SQL入門	5500	山田出版	5500.0000
PHPドリル	5100	山田出版	5300.0000
XML辞典	3300	翔泳社	4633.3333
XMLリファレンス	3200	日経BP	3866.6667
PHP5サンプル集	3000	秀和システム	3166.6667
PEAR入門	3000	翔泳社	3066.6667
SQLリファレンス	2500	日経BP	2833.3333
JSPリファレンス	1800	秀和システム	2433.3333
SQLプチブック	1600	日経BP	1966.6667
らくだの観察日記	1100	山田出版	1500.0000
フェレットの観察	1000	山田出版	1233.3333
ハムスターの観察	900	山田出版	1000.0000
かえるの観察日記	800	山田出版	900.0000
あひるの観察日記	700	山田出版	800.0000

← (5500) / 1

← (5500 + 5100) / 2

← (5500 + 5100 + 3300) / 3

← (5500 + 5100 + 3300 + 3200) / 3 ※※

⋮

⋮

赤い線がフレームとなり、その中で平均したものを3866.6667として出力する

FOLLOWINGを利用する

自分より一つ前、自分、自分より一つ先を集計対象とする

```
-- FOLLOWINGを使った例
SELECT
 title,
 price,
 publish,
 SUM(price) OVER (ORDER BY price DESC) AS '累計',
 AVG(price) OVER (ORDER BY price DESC
 ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS '移動平均'
-- ROWS BETWEEN ここから PRECEDING AND ここまで FOLLOWINGみたいな書き方で、ここから〜ここまでの平均をとる。BETWEENがカレントレコードからどこまで前か、FOLLOWINGがカレントレコードからどこまで先かを表している
FROM
 books
;
```

ウィンドウ関数一覧

- 集約関数(SUM, AVG, COUNT, MAX, MIN)など
- RANK, DENSE\_RANK, ROW\_NUMBERなど

ユーザの追加

hogeというユーザを作成

```
-- hogeというユーザをパスワード「abcdefg」で作成する
CREATE USER hoge IDENTIFIED BY 'abcdefg';
```

ユーザの一覧を表示するには

```
SELECT
 User
FROM
 mysql.user
;
```

権限を与える

```
-- workbookから始まるテーブルへのSELECT権限を与えることができる
GRANT SELECT ON workbook.* TO hoge;
```

権限を削除

```
REVOKE SELECT ON workbook.* FROM hoge;
```

アカウントを削除

```
DROP USER hoge;
```

## MySQLの認証方式について

今までmysql\_native\_passwordという認証方式を使っていた。

認証方式とは・パスワードの認証方式をどのようにするのか？

- caching\_sha2\_passwordは、MySQL8.0のデフォルトの認証方式
- mysql\_native\_passwordは、古い方式。セキュリティが低く、MySQL8.4からは禁止されている

```
-- ユーザにどの認証方式を使ってもらうかを指定する方法
-- これは中でハッシュ化して保存されているからおすすめ
CREATE USER hoge IDENTIFIED WITH caching_sha2_password BY 'abcdefg'
```

## パスワードに有効期限を設定する

hogeアカウントのパスワードの有効期限を90日に設定

```
ALTER USER hoge PASSWORD EXPIRE INTERVAL 90 DAY;
```

hogeアカウントのパスワードを無期限に設定

```
ALTER USER hoge PASSWORD EXPIRE NEVER;
```

強制的にパスワードを期限切れにする

```
ALTER USER hoge PASSWORD EXPIRE;
```

## ロールについて

ロールとは、ユーザの設定が含まれたユーザのようなもの。イメージするといえば、いろんな初期設定がされている初期スキンみたいなもの

```
CREATE ROLE app_user; --app_userというロールを作成
GRANT SELECT ON workbook.* TO app_user; --ロールにworkbook系のselect権限を付与する
GRANT INSERT ON workbook.books TO app_user; -- ロールにbooksの挿入権限を付与する
CREATE ROLE app_dev; -- app_devというロールを作成する
GRANT ALL ON workbook.* TO app_dev; -- workbookの全ての権限(パーミッション)をapp_devに付与
する

-- user1にapp_userロールの設定を着させてあげる
GRANT app_user TO user1;

-- app_userの設定されたロールを有効化する
SET ROLE app_user;

-- app_userに設定されたロールを無効にし、
SET ROLE app_dev;

-- ロールを完全に無効化する
SET ROLE NONE;
```

## ボイスコッド正規系（第三正規系のさらに上）

第三を超える正規系のことを高次正規系という。



非キーから主キーへの関数従属がある。例：チーム補佐（123W）といえば、チーム（001）だよね。的なものがあると、それはボイスコード正規系とは言えないらしい。

つまり、ボイスコード正規系には、非キーから主キーへの関数従属がないものをいう。

## ■ボイスコード正規形を満たしていないテーブル

### 社員 - チーム - リーダー

社員 ID	チームコード	チーム補佐
000A	001	123W
000B	001	456Z
000B	002	003O
001F	001	123W
001F	002	003O
003O	002	999Y

ボイスコード正規系じゃなかったらどんな問題が起きるのか？

- チーム補佐が担当チームを変える場合に複数行の更新が発生する。例：001チームが、チームを変更する場合、そのチーム補佐をレコード単位で一つずつ変えていかないとけなくなる
- 社員がチームに参加するまで、チーム補佐が登録できない
- 社員がチームを外れた場合、レコードを削除すると、チーム補佐がこのチームを担当していたよ～という内容がなくなってしまう。

## ボイスコード正規系手順

### 1. テーブルを分離

※この場合、社員ID(000A)さんの補佐が123Wさんと456Zさんの二人が存在するということになってしまう。



## ■「社員-チーム-リーダー」テーブルを分解

社員 - チーム

社員 ID	チームコード
000A	001
000B	001
000B	002
001F	001
001F	002
003O	002

チーム補佐 - チーム

チーム補佐	チームコード
123W	001
456Z	001
003O	002
999Y	002

### 2.ボイスコード正規系がここの手順

※この場合、社員-チーム補佐テーブルに、(000A,003O)というレコードを打つことができる。しかし、そうすると、000Aは補佐123Wが担当しているので000Aはチーム001である。しかし上記のレコードを打つと、000Aの補佐は003Oであり、003Oは別チーム（002）に所属しているので、000Aは不意に002チームに所属してしまうことになる。

## ■ボイスコード正規形

社員 - チーム補佐

社員 ID	チーム補佐
000A	123W
000B	456Z
000B	003O
001F	123W
001F	003O
003O	999Y

チーム補佐 - チーム

チーム補佐	チームコード
123W	001
456Z	001
003O	002
999Y	002

## 第四正規系

### 1.間に関連エンティティを入れてあげる

※こうすれば、社員IDひとつがどのチームに所属しているかはっきりさせることができる。このように間で補助をしてくれるテーブルを関連エンティティという

■第3正規形

社員 - チーム

社員 ID	チームコード
000A	001
000B	001
000B	002
001F	001
001F	002
003O	002

2. 関連エンティティでの問題

※この場合、チーム001が002に移動する場合、現実的には可能だが、複数行に対して直さないといけなくなる。

■第4正規形  
社員 - チーム - 製品

社員 ID	チームコード	製品コード
000A	001	P1
000A	001	P2
000B	001	P1
000B	002	P1
001F	001	P2
001F	002	P2
003O	002	P2
003O	002	P3

社員「000A」は「P1」と「P2」  
の二つの製品を担当

3. 第四正規系にする



## ■第4正規形

### 社員 - チーム

社員 ID	チームコード
000A	001
000B	001
000B	002
001F	001
001F	002
003O	002

### 社員 - 製品

社員 ID	製品コード
000A	P1
000B	P1
001F	P2
003O	P2
003O	P3

## 第五正規系

この場合、チーム〇〇がどの製品を作っているか、シンプル化されていない

※もしも、チーム001がP3も担当していた場合、ここでは把握できない

## ■第4正規形（再掲）

### 社員 - チーム

社員 ID	チームコード
000A	001
000B	001
000B	002
001F	001
001F	002
003O	002

### 社員 - 製品

社員 ID	製品コード
000A	P1
000B	P1
001F	P2
003O	P2
003O	P3

分けて、これといえこれ！という形を作ってあげる（第五正規系）

## ■第5正規形

社員 - チーム

社員 ID	チームコード
000A	001
000B	001
000B	002
001F	001
001F	002
003O	002

社員 - 製品

社員 ID	製品コード
000A	P1
000B	P1
001F	P2
003O	P2
003O	P3

チーム - 製品

チームコード	製品コード
001	P1
001	P2
002	P1
002	P2
002	P3

## rubyでダミーユーザ

### 1.gemをインストール

```
gem install securerandom
```

### ダミー作成

```
まず、Fakerをインストールしていない場合は、Gemfileに以下を追加して `bundle install` を実行してください。
gem 'faker'

require 'faker'
require 'securerandom' # パスワードのハッシュ化（簡略版）に利用

--- Fakerのロケールを日本語に設定 ---
Faker::Config.locale = 'ja'

--- テーブル名とカラム名の設定 ---
users テーブルに 'address' カラムが追加されていることを想定
TABLE_NAME = "users"
COLUMNS = %w[name email password_digest address created_at updated_at]

--- ヘルパー関数（現在時刻のフォーマット） ---
def current_mysql_datetime
 Time.now.strftime('%Y-%m-%d %H:%M:%S')
end

puts "-- MySQL INSERT Statements for #{TABLE_NAME} table"
puts "-- Generated on #{current_mysql_datetime}"
puts ""

10人分のダミーデータを生成し、INSERT文を出力
10.times do |i|
```

```

ダミーデータの生成
日本人の漢字の氏名を生成 (苗字 + 名前)
last_name = Faker::Name.last_name # 姓
first_name = Faker::Name.first_name # 名
name = "#{last_name} #{first_name}" # フルネーム

確実なアルファベットのメールアドレスを生成
Faker::Internet.username はロケールに依存せずアルファベット文字列を生成します。
これに固定のドメインを組み合わせることで、漢字を含まないメールアドレスを保証します。
ユニーク性を確保するため、Faker::Internet.unique を使用します。
begin
 email_username = Faker::Internet.unique.username(specifier: 8..15) # 8~15文字
 のランダムなアルファベットユーザー名
 email_domain = Faker::Internet.domain_name # 例: example.com, test.org など
 email = "#{email_username}@#{email_domain}"
rescue Faker::UniqueGenerator::LimitExceededException
 # 万が一、ユニークなユーザー名が枯渇した場合のフォールバック
 email = "fallback_user_#{i}_#{SecureRandom.hex(4)}@example.com"
end

パスワードは実際のRailsアプリケーションではbcryptなどでハッシュ化されますが、
ここでは簡略化のためにランダムな文字列を生成しています。
実際の環境では本物のハッシュ化したパスワードを使用してください。
password_digest = SecureRandom.hex(10) # 例: ランダムな20文字の16進数文字列

日本の住所を生成 (都道府県、市区町村、番地、建物名など)
address = "#{Faker::Address.state}#{Faker::Address.city}#{
Faker::Address.street_address} #{Faker::Address.building_number}"
オプションで郵便番号も追加したい場合
address = "〒#{Faker::Address.zip_code} #{Faker::Address.state}#{
Faker::Address.city}#{Faker::Address.street_address} #{
Faker::Address.building_number}"

created_at = current_mysql_datetime
updated_at = current_mysql_datetime

VALUES句の値をエスケープして文字列として準備
MySQLの文字列リテラルはシングルクォートで囲み、内部のシングルクォートはエスケープする必要がある
escaped_name = name.gsub("'", "'") # gsubとは、置換メソッドで、左のは、シングルクォートを見つ
けたときにシングルクォートを二つ付ける処理をしている
escaped_email = email.gsub("'", "'")
escaped_password_digest = password_digest.gsub("'", "'")
escaped_address = address.gsub("'", "'")

INSERT文の生成
insert_statement = <<-SQL
INSERT INTO #{TABLE_NAME} (#{COLUMNS.join(', ')})
VALUES (
 '#{escaped_name}',
 '#{escaped_email}',
 '#{escaped_password_digest}',
 '#{escaped_address}',
 '#{created_at}',
 '#{updated_at}'

```

```
);
SQL

puts insert_statement
end

puts ""
puts "-- End of INSERT statements"
```

## EXPLAINとは

クエリの実行計画を確認するためのツール

```
EXPLAIN SELECT *
FROM
 books
WHERE
 title = "かえるの観察日記"
;
```

```
+---+-----+-----+-----+-----+-----+-----+-----+
----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len |
ref | rows | filtered | Extra |
+---+-----+-----+-----+-----+-----+-----+-----+
----+-----+-----+-----+-----+
| 1 | SIMPLE | books | NULL | ALL | NULL | NULL | NULL |
NULL | 14 | 10.00 | Using where |
+---+-----+-----+-----+-----+-----+-----+
----+-----+-----+-----+-----+
```

- id ... クエリ識別子
- select\_type ... クエリの種類(SIMPLE, PRIMARYなど)

SIMPLE ... サブクエリやUNIONを含まない単純なSELECT文  
PRIMARY ... サブクエリやUNIONを含むSELECT文

- table ... 対象のテーブル名
- partitions ... 使用されるパーティション

補足：パーティションとは？  
グループ化される際のキーとなるカラム名のこと

- type ... アクセスの方法(ALL , index, range, refなど)

ALL ... 最も遅く、片っ端から一行ずつチェックする  
index ... インデックスを使ったフルスキャンをする  
range ... BETWEENなどを使い、特定の範囲だけの行をフルスキャンする  
ref ... indexで複数行を絞り込みする

- possible\_keys ... クエリで使用可能なインデックス候補
- key ... 実際に使われたインデックス
- key\_len ... インデックスの長さ (バイト数)
- ref ... インデックスと比較される値やカラム
- rows ... 処理対象の行数
- filtered ... 絞り込まれた行の全体に対する割合
- Extra ... 補足情報